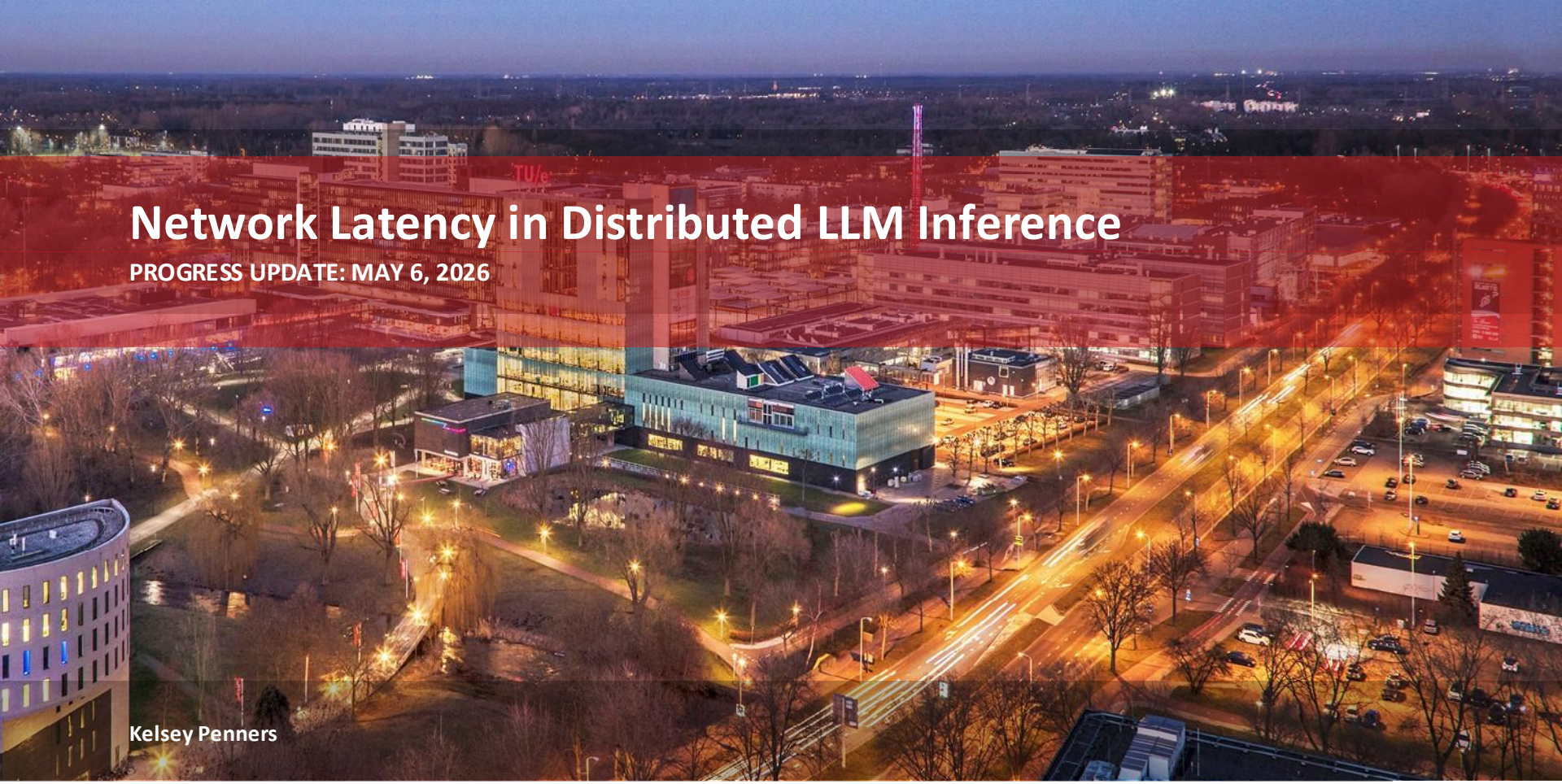


Network Latency in Distributed LLM Inference

PROGRESS UPDATE: MAY 6, 2026

Kelsey Penners

Recap & Progress

Multi GPU Setup

Model: Llama 3.1-8B (baseline)

Servers: shyfec (NVLink, SHM, Network Socket) + funnotch (PCIe, SHM, Socket)

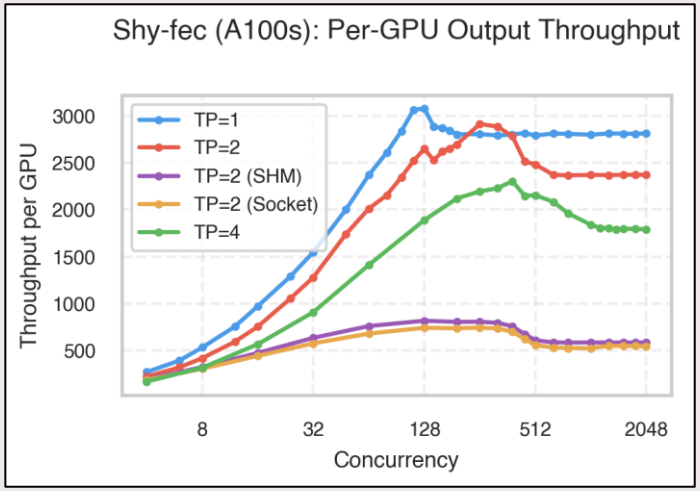
TP configurations: 1, 2, 4 on A100s and V100s

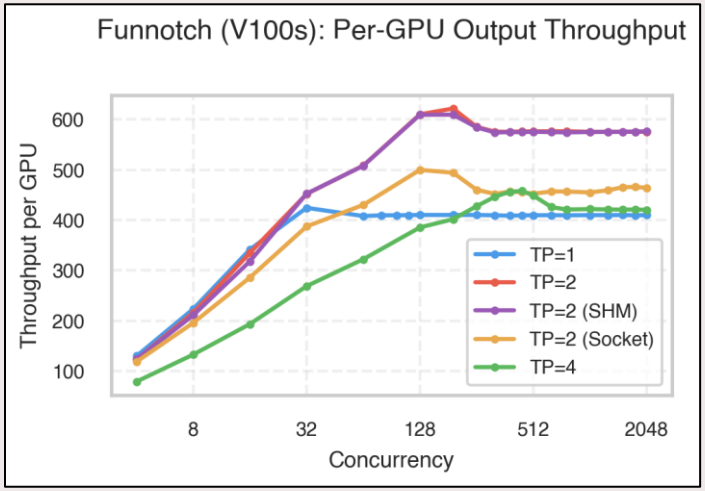
Workload: ISL/OSL = 512/512 (interactive chat load) over concurrent requests

Progress

- Performance data from V100 (funnotch)
- SLO-constrained capacity / max throughput analysis

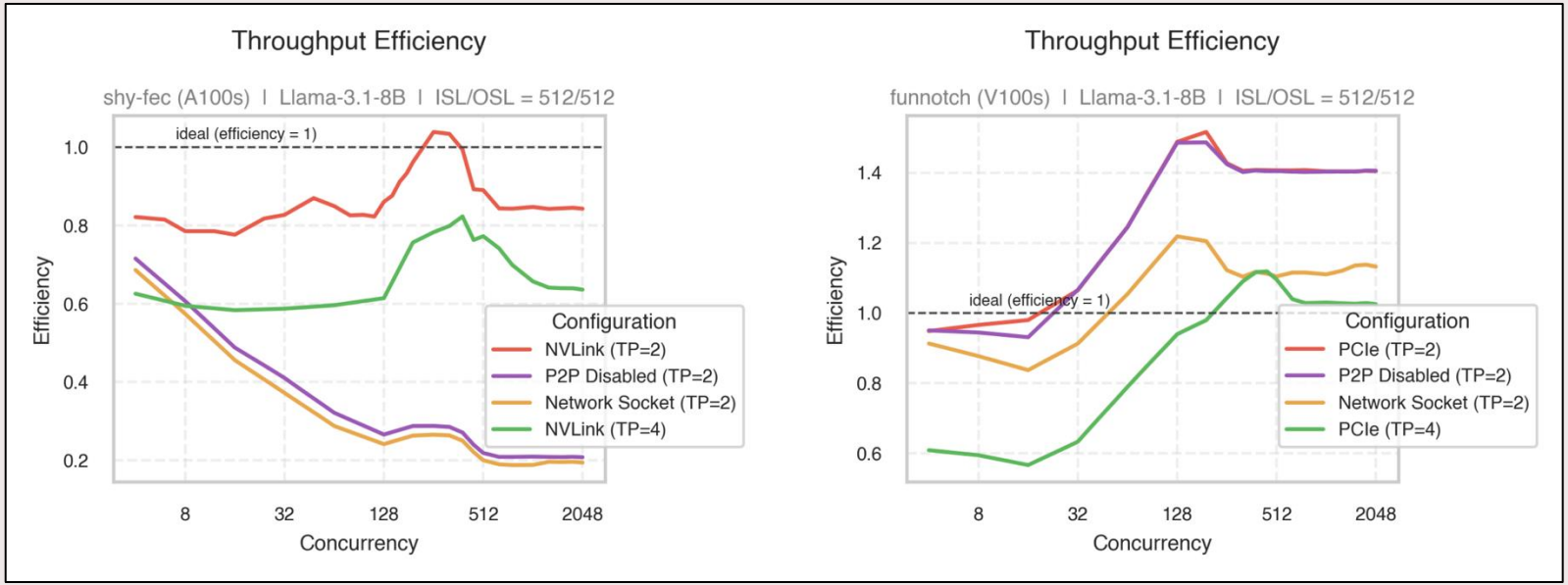
Scaling on A100 vs V100 GPUs





- V100 PCIe and P2P Disabled are identical: P2P is only active between a single pair of GPUs on funnotch.
- A100 SHM transport uses PCIe, performance similar to V100 PCIe/P2P disabled.

Throughput Efficiency Across Servers



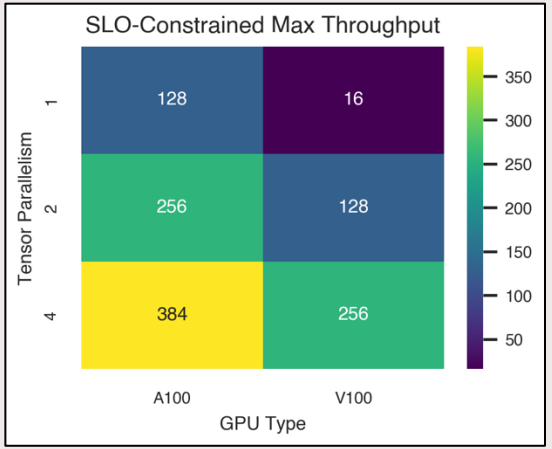
V100-PCIe GPUs show better scaling throughput efficiency than A100s over NVLink

SLOs & Maximizing Performance

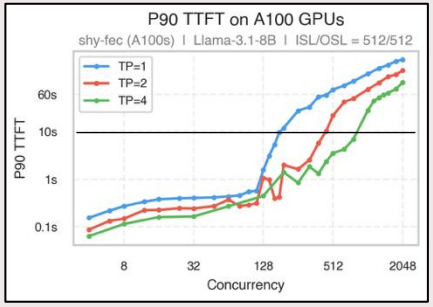
- Service Level Objective (SLO): target for a metric
  - Depend heavily on product type. Interactive chat requires fast TTFT and ITL, Offline inference depends more on throughput, E2EL can vary
  - Maximum capacity is highest concurrency where SLOs are satisfied, but this doesn't always maximize performance…

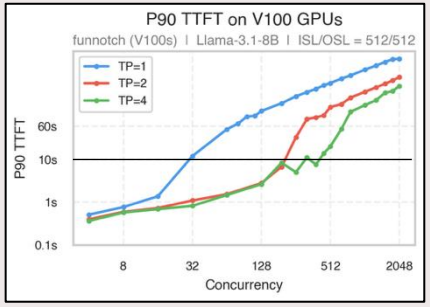
Typical SLO Thresholds:

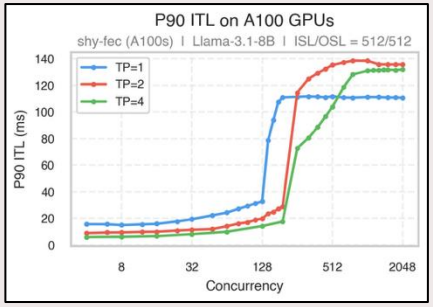
- TTFT: 500ms – 2s
- ITL: 100-300ms
  - (170ms generates tokens at average reading speed)
- E2EL: 10-30s (depends on output length)

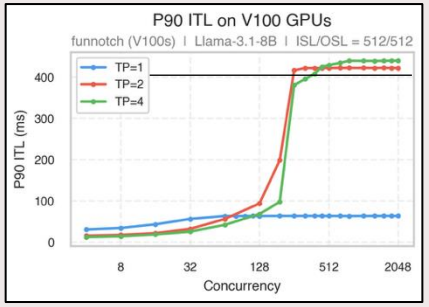


SLOs



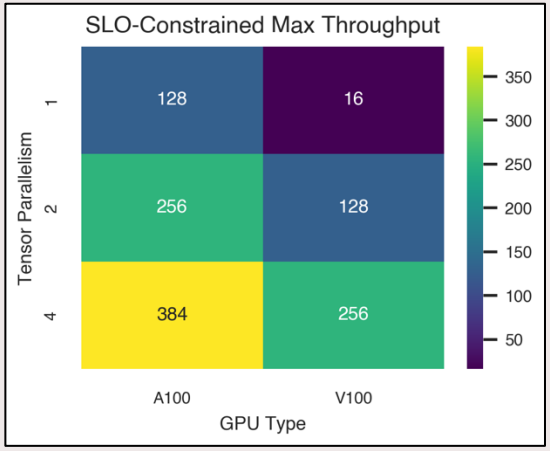






SLO Thresholds:

- P90 TTFT < 10s
- P90 ITL < 400ms

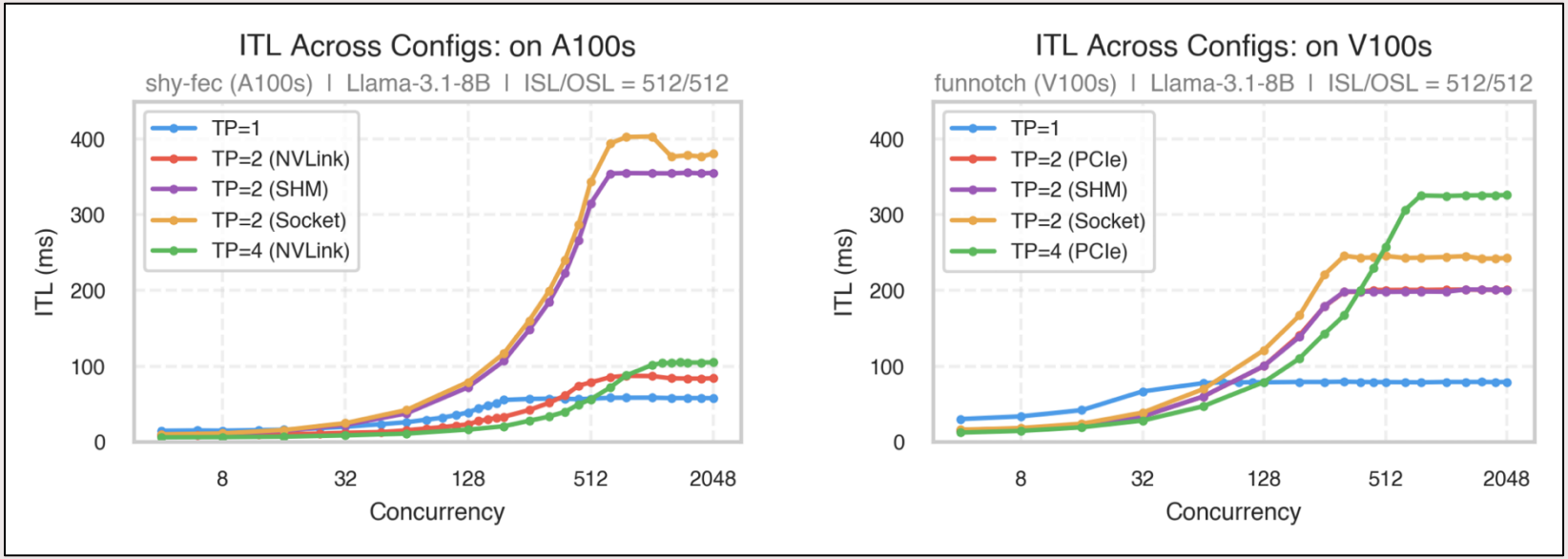


Next Steps

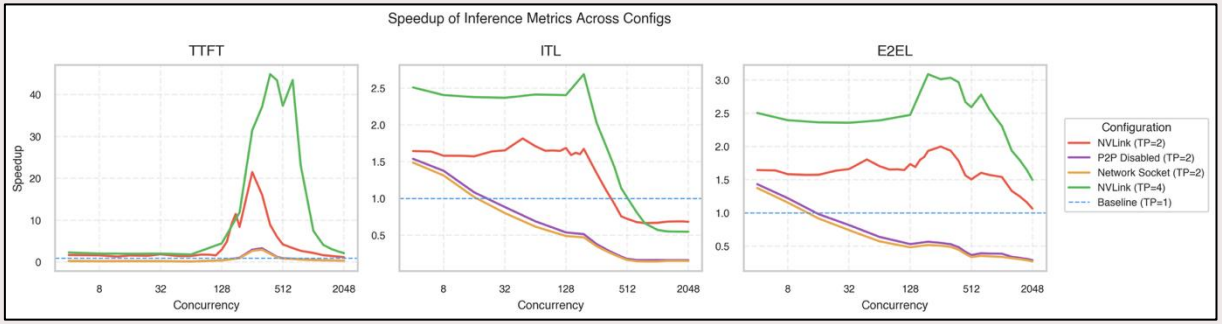
- Model scope
- Latency injection?
- Explore other workloads?
- Writing & citing

Thank you!

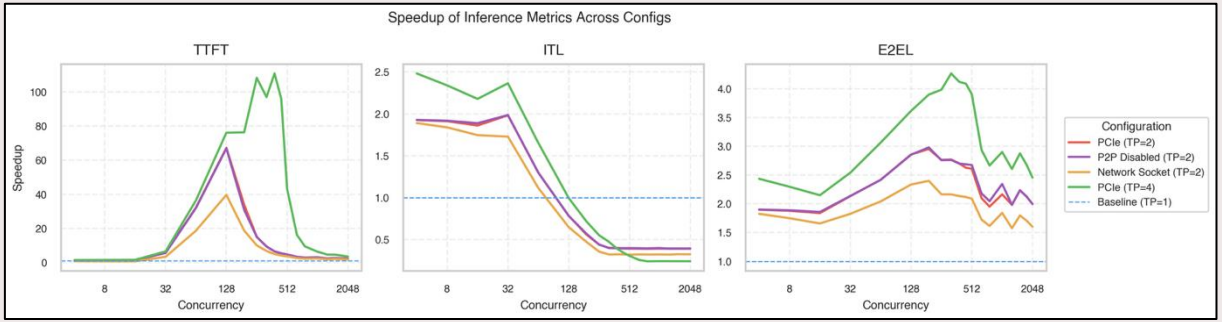
Inter-Token Latency Across Configs



Latency Metrics Speedup



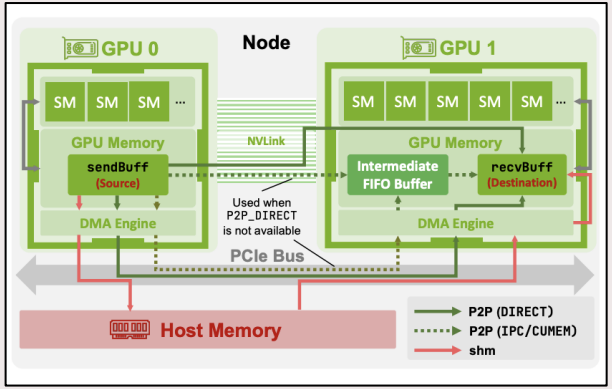
On A100 GPUs (shy-fec)



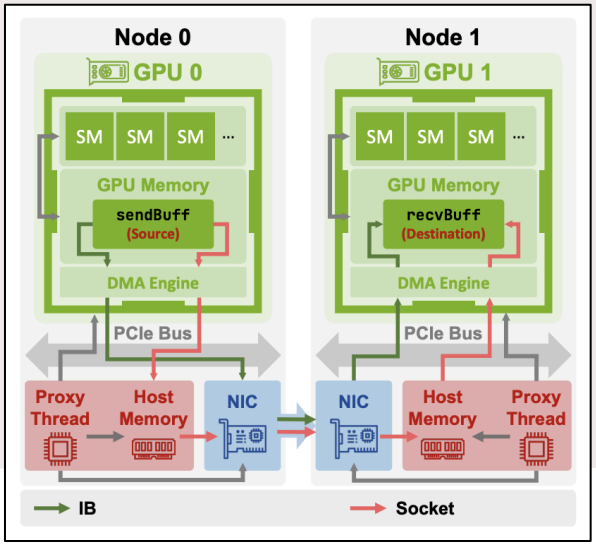
On V100 GPUs (funnotch)

NCCL Transport

- NCCL prioritizes lowest latency, highest bandwidth paths available (often P2P)
- Shared memory (SHM):
  - Data routed via system memory
  - GPU → PCIe → Host Mem → PCIe → GPU
- Inter-node communication:
  - Managed by CPU proxy thread
  - Uses TCP sockets or Infiniband
- Socket-based communication:
  - Multiple memory copies
  - GPU → PCIe → Host Mem → TCP stack → loopback → TCP stack → Host Mem → PCIe → GPU.



source: https://arxiv.org/pdf/2507.04786v1



Model of Communication Overhead

$$\mathrm{ITL}_{\mathrm{TP=k}} = \mathrm{ITL}_{\mathrm{compute}} + 64 * \mathrm{L}_{\mathrm{AllReduce}}(k)$$

$$\mathrm{ITL}_{\mathrm{compute}} \approx \mathrm{ITL}_{\mathrm{TP=1}}$$

$$\Delta\mathrm{ITL} = \mathrm{ITL}_{\mathrm{TP=k}} - \mathrm{ITL}_{\mathrm{compute}}$$

$$\mathrm{L}_{\mathrm{observedAllReduce}} = \Delta\mathrm{ITL} \,/\, 64$$

Taking low concurrency,TP=1 as approximately pure compute ITL, we can estimate AllReduce latency. Very oversimplified, as AllReduce latency varies due to message size, and ITL also depends on sequence length.